

React Engineering Handbook: Mastering React through BiteBuddy

This guide breaks down core React concepts using the architecture, design patterns, and code of your BiteBuddy application.

1. React Architecture

Theory

React uses a Component-Based Declarative Architecture that builds interfaces through modular, isolated, and reusable units of code. Unlike imperative architectures (like vanilla JavaScript, where you tell the browser exactly how to update the DOM step-by-step), React operates declaratively. You define what the UI should look like for a given state, and React handles syncing the DOM.

It enforces a unidirectional (one-way) data flow where data flows down from parent components to child components via read-only props.

Application in BiteBuddy

In BiteBuddy, the architecture is managed by App.jsx. It serves as the State Colocation Root and Router.

```
function App() { const [currentPage, setCurrentPage] = useState('dashboard'); const [profile, setProfile] =
useState(null); const [activePlan, setActivePlan] = useState(null); ... return ( <div className="app-layout">
<Sidebar currentPage={currentPage} navigateTo={setCurrentPage} /> <main className="app-content"> {renderPage()}
</main> <MobileNav currentPage={currentPage} navigateTo={setCurrentPage} /> </div> ); }
```

Here, App.jsx acts as the single orchestrator. It fetches user profiles, manages layouts, and controls the navigation routes (Sidebar, MobileNav, and the dynamic main content).

Problem Solved & What Happens if Removed

- Problem Solved: Prevents state synchronization bugs. If page states, active meal plans, and user profiles were scattered across individual files, updating the diet type in Goals.jsx wouldn't automatically update the nutritional targets in Dashboard.jsx.
 - If Removed: The application would revert to jQuery-style DOM updates. You would have to write manual selector queries (document.querySelector) and parse variables across windows, leading to fragile code and synchronization errors.
-

2. Components

Theory

Components are the building blocks of a React application. They are independent, reusable pieces of code that return HTML-like structures (JSX). Think of them as custom HTML tags that encapsulate their own structure, styling, and behavior.

Application in BiteBuddy

BiteBuddy distinguishes between Smart/Container Components (like Dashboard.jsx) and Dumb/Presentational Components (like Card.jsx).

```
import React from 'react'; import './Card.css'; export const Card = ({ children, className = '', ...props }) => {
  return ( <div className={`bb-card ${className}`} {...props}> {children} </div> ); };
```

This presentational component is highly reusable. It has no internal logic, API calls, or state; it simply wraps incoming layout elements (children) with standard CSS styles (bb-card).

Problem Solved & What Happens if Removed

- Problem Solved: Prevents code duplication. Instead of writing CSS styles and structural code for every card container across the Dashboard, Food Scanner, and Workout Log pages, we write it once in Card.jsx.
- If Removed: You would have to manually add CSS classes and duplicate structural containers for every content block, making layout updates much harder to coordinate.

3. JSX (JavaScript XML)

Theory

JSX is a syntax extension for JavaScript that allows you to write HTML-like elements directly in your JS files. Browsers cannot read JSX directly; it is compiled during build time by tools like Babel or ESBuild into standard React.createElement() function calls.

Application in BiteBuddy

In WeeklyChart.jsx, we write HTML-like structures mixed with JavaScript dynamic evaluations:

```
return ( <div className="weekly-chart" style={{ height }}> <div className="weekly-chart-bars"> {data.map((item,
index) => { const heightPercent = Math.min((item.value / maxValue) * 100, 100); return ( <div key={index}
className="weekly-chart-col"> <div className="weekly-chart-bar-container"> <div className={`weekly-chart-bar`}
style={{ height: `${heightPercent}%` }} /> </div> </div> ); }} } </div> </div> );
```

Problem Solved & What Happens if Removed

- Problem Solved: Simplifies writing component structures. It allows you to build dynamic UIs with native JavaScript controls (like .map() loops and ternary operators) rather than using complex string concatenations or custom template syntaxes.
- If Removed: You would have to write raw JavaScript calls to update the DOM, such as React.createElement, making writing and maintaining nested UI trees significantly harder.

4. Functional Components

Theory

Functional components are JavaScript functions that accept props as an argument and return a React element (JSX). They are the modern standard in React development, replacing legacy Class Components by utilizing Hooks to manage state and lifecycle events.

Application in BiteBuddy

Every component in BiteBuddy is built as a functional component. For example, Sidebar.jsx:

```
export const Sidebar = ({ isCollapsed, toggleCollapse, currentPage, navigateTo }) => { return ( <aside  
  className={`bb-sidebar ${isCollapsed ? 'collapsed' : ''}`> {/* Navigation Layout */} </aside> ); };
```

5. Props (Properties)

Theory

Props are read-only configuration inputs passed from parent components to child components. They are immutable; a component should never modify its own props. Updates to child props must be initiated by the parent component changing the values it passes down.

Application in BiteBuddy

In App.jsx, state values and setter functions are passed down to Shopping.jsx as props:

```
<Shopping profile={profile} shopping={shoppingItems} setShoppingItems={setShoppingItems}  
checkedItems={checkedItems} setCheckedItems={setCheckedItems} API_BASE={API_BASE} />
```

6. State

Theory

State is an object managed internally by a component that stores dynamic data. Unlike props (which are passed down and immutable), state is local, mutable, and private to the component. When state values update, React automatically triggers a re-render of the component and its children.

Application in BiteBuddy

In WorkoutLog.jsx, state tracks whether the input form modal is open:

```
export const WorkoutLog = ({ API_BASE }) => { const [logs, setLogs] = useState([]); const [loading, setLoading] =  
  useState(true); const [showForm, setShowForm] = useState(false); // Local UI state ... };
```

7. useState

Theory

useState is a React Hook that adds state variables to functional components. It accepts the initial state value as its only argument and returns an array containing two elements: the current state value and a setter function to update it.

Application in BiteBuddy

In FoodScanner.jsx, useState manages uploaded image states:

```
export const FoodScanner = () => { const [selectedImage, setSelectedImage] = useState(null); const [isScanning, setIsScanning] = useState(false); const [scanResult, setScanResult] = useState(null); ... };
```

8. useEffect

Theory

useEffect lets functional components run side effects, such as API calls, subscriptions, and DOM updates. It accepts a callback function containing the side effect logic and an optional dependency array.

Application in BiteBuddy

In FoodLog.jsx, useEffect runs once when the component mounts to fetch today's logged meals:

```
export const FoodLog = ({ API_BASE, navigateTo }) => { const [logs, setLogs] = useState([]); useEffect(() => { fetchLogs(); }, []); // Runs once on mount };
```

9. useCallback

Theory

useCallback returns a memoized version of a callback function, caching its reference between renders. It only recreates the function if one of the dependencies in its dependency array changes.

Application in BiteBuddy

In AICoach.jsx, the sendMessage function is memoized to prevent recreating it on every user keystroke:

```
export const AICoach = () => { const [messages, setMessages] = useState([]); const [input, setInput] = useState(''); const sendMessage = useCallback(async (text) => { const msgText = text || input.trim(); if (!msgText) return; ... }, [input, messages]); };
```

10. useRef

Theory

useRef returns a mutable ref object whose .current property is initialized with the passed argument. This ref persists for the lifetime of the component. It can reference DOM nodes directly.

Application in BiteBuddy

In AICoach.jsx, useRef is used to automatically scroll the chat window to the bottom when new messages arrive:

```
export const AICoach = () => { const messagesEndRef = useRef(null); // Reference to DOM element const
scrollToBottom = () => { messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' }); }; ... };
```

11. Controlled Components

Theory

A controlled component binds its form input value to React state. React acts as the "single source of truth." The input value is read from state, and updates are handled by an onChange callback.

Application in BiteBuddy

In Onboarding.jsx, form fields are controlled using a single formData state object:

```
export const Onboarding = ({ API_BASE, onComplete }) => { const [formData, setFormData] = useState({ name: '', age:
'', gender: 'male', height: '' }); const handleChange = (e) => { setFormData({ ...formData, [e.target.name]:
e.target.value }); }; return ( <Input name="name" value={formData.name} onChange={handleChange} /> ); };
```

12. Event Handling

Theory

React handles events using camelCase attributes (like onClick or onChange). It wraps browser events in a cross-browser wrapper called SyntheticEvent.

Application in BiteBuddy

In Shopping.jsx, click events use stopPropagation() to prevent check triggers when clicking edit or delete buttons:

```
const removeShoppingItem = (idx, e) => { e.stopPropagation(); // Prevents checking the item when clicking delete
const currentItems = [...shopping]; currentItems.splice(idx, 1); setShoppingItems(currentItems); };
```

13. Conditional Rendering

Theory

Conditional rendering allows you to display different UI elements based on state values or conditions. It uses standard JavaScript syntax, such as ternary operators and logical AND operators (&&).

Application in BiteBuddy

In FoodScanner.jsx, conditional rendering displays either the upload zone or the image preview:

```
return ( <div className="scanner-grid"> <div className="scanner-main"> {!selectedImage ? ( <Card
  className="upload-zone"> { /* File input selectors */} </Card> ) : ( <Card className="preview-zone"> { /* Image
  preview */} </Card> )} </div> </div> );
```

14. Lists and Keys

Theory

When rendering lists of elements in React, you must assign a unique key prop to each item. React uses these keys during Virtual DOM reconciliation to track which items have changed.

Application in BiteBuddy

In WeeklyChart.jsx, keys are assigned to dynamically generated chart columns:

```
<div className="weekly-chart-bars"> {data.map((item, index) => ( <div key={index} className="weekly-chart-col"> { /*
  Bar rendering */} </div> ))} </div>
```

15. Forms

Theory

React forms use controlled components to capture, validate, and submit user inputs, calling `e.preventDefault()` to prevent page reloads.

Application in BiteBuddy

In WorkoutLog.jsx, the workout form is submitted using an event handler:

```
const submitLog = async () => { try { const payload = { ...formData, exercises: formData.exercises.split(',').map(s
=> s.trim()).filter(s => s) }; const res = await fetch(`${API_BASE}/workouts`, { method: 'POST', headers: {
  'Content-Type': 'application/json' }, body: JSON.stringify(payload) }); if (!res.ok) throw new Error('Failed to log
workout'); } catch (e) { addToast('Error', e.message, 'error'); } };
```

16. API Calls using fetch()

Theory

The Fetch API is a built-in browser interface for fetching resources across networks using JavaScript Promises.

Application in BiteBuddy

In App.jsx, fetch is used to load the user's active profile and meal plan when the application mounts:

```
const loadData = async () => { setLoadingProfile(true); try { const profRes = await fetch(`${API_BASE}/profile`);  
  if (profRes.ok) { const profData = await profRes.json(); setProfile(profData); } } catch (e) {  
    console.error("Failed to load backend data", e); } finally { setLoadingProfile(false); } };
```

17. Async/Await

Theory

async/await is a syntactic wrapper around JavaScript Promises, making asynchronous code read like synchronous code.

Application in BiteBuddy

In MealPlanner.jsx, asynchronous calls fetch recipe instructions from the chatbot backend:

```
const generateRecipe = async (mealName) => { setIsGenerating(true); try { const res = await  
  fetch(`http://localhost:8000/api/chat`, { method: 'POST', headers: { 'Content-Type': 'application/json' }, body:  
    JSON.stringify({ messages: [{ role: 'user', content: `Can you give me a simple recipe for ${mealName}?` }] }) });  
  const data = await res.json(); setGeneratedRecipe(data.response); } catch { addToast('Error', 'Could not generate  
  recipe.', 'error'); } finally { setIsGenerating(false); } };
```

18. Loading States

Theory

Loading states tell the user that an asynchronous task is in progress, preventing duplicate triggers and improving user experience.

Application in BiteBuddy

In App.jsx, a loading state is displayed while fetching user profile records:

```
if (loadingProfile) { return <div className="app-loading">Loading BiteBuddy...</div>; }
```

19. Error Handling

Theory

Error handling ensures that application failures (such as server timeouts) are caught and handled gracefully, preventing the application from crashing.

Application in BiteBuddy

BiteBuddy uses try-catch blocks for API calls and displays notifications using the useToast hook.

20. Component Communication

Theory

React uses unidirectional data flow. Parent-to-Child communication occurs via props, and Child-to-Parent occurs using callback functions.

Application in BiteBuddy

In App.jsx, callbacks like setShoppingItems are passed to Shopping.jsx to sync list changes.

21. Folder Structure

Theory

A clean folder structure keeps the codebase organized and scalable by grouping files by feature or by type.

22. CSS Organization

Theory

CSS organization ensures styles are easy to manage and don't conflict, using global CSS variables and local stylesheets.

23. Re-rendering

Theory

Re-rendering is the process where React calls a component function to calculate updated layout structures in response to state or prop updates.

24. Virtual DOM

Theory

The Virtual DOM is a lightweight, in-memory representation of the real DOM. React runs a diffing algorithm to update only the changed nodes in the real DOM.

25. React Lifecycle (using hooks)

Theory

Functional components use Hooks like `useEffect` to manage component mounting, updating, and unmounting lifecycles.

26. Performance Optimization

Theory

Optimization focuses on skipping unnecessary updates using caching techniques like `React.memo`, `useMemo`, and `useCallback`.

27. Reusable Components

Theory

Reusable components are presentation elements that accept props to render layouts consistently (e.g. `ProgressBar`).

28. React Best Practices

Theory

Guidelines include component focus, localizing state, avoiding direct DOM access, and handling error exceptions.

29. 15 KEY REACT VIVA QUESTIONS

1. What is the difference between State and Props?

Props are read-only inputs passed from a parent component, while state is local, mutable data managed internally by the component.

2. What triggers a component re-render in React?

Updates to local state, new props received from parent components, or updates to active React Contexts.

3. Why are keys required in dynamic lists?

Keys help React's Virtual DOM identify which list items have changed, been added, or been removed, ensuring efficient updates.

4. What is the Virtual DOM?

A lightweight, in-memory representation of the real DOM. React runs a diffing algorithm to update only the modified nodes.

5. Is useState synchronous?

No, state updates in React are asynchronous and batched for performance.